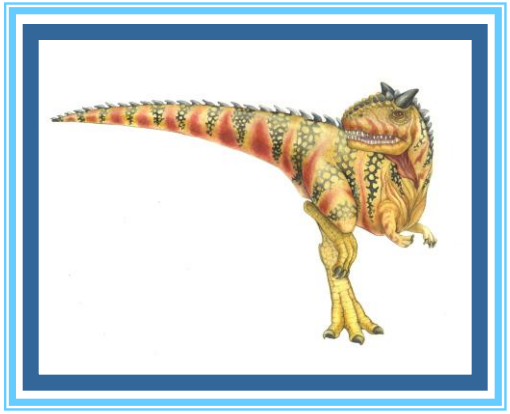
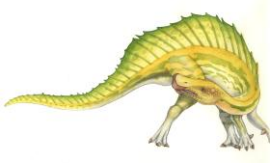
Chapter 2: Operating-System Structures



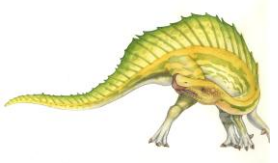
Chapter 2: Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Structure
- Virtual Machines
- System Boot



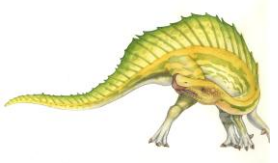
Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device



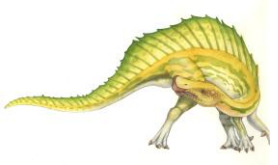
Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

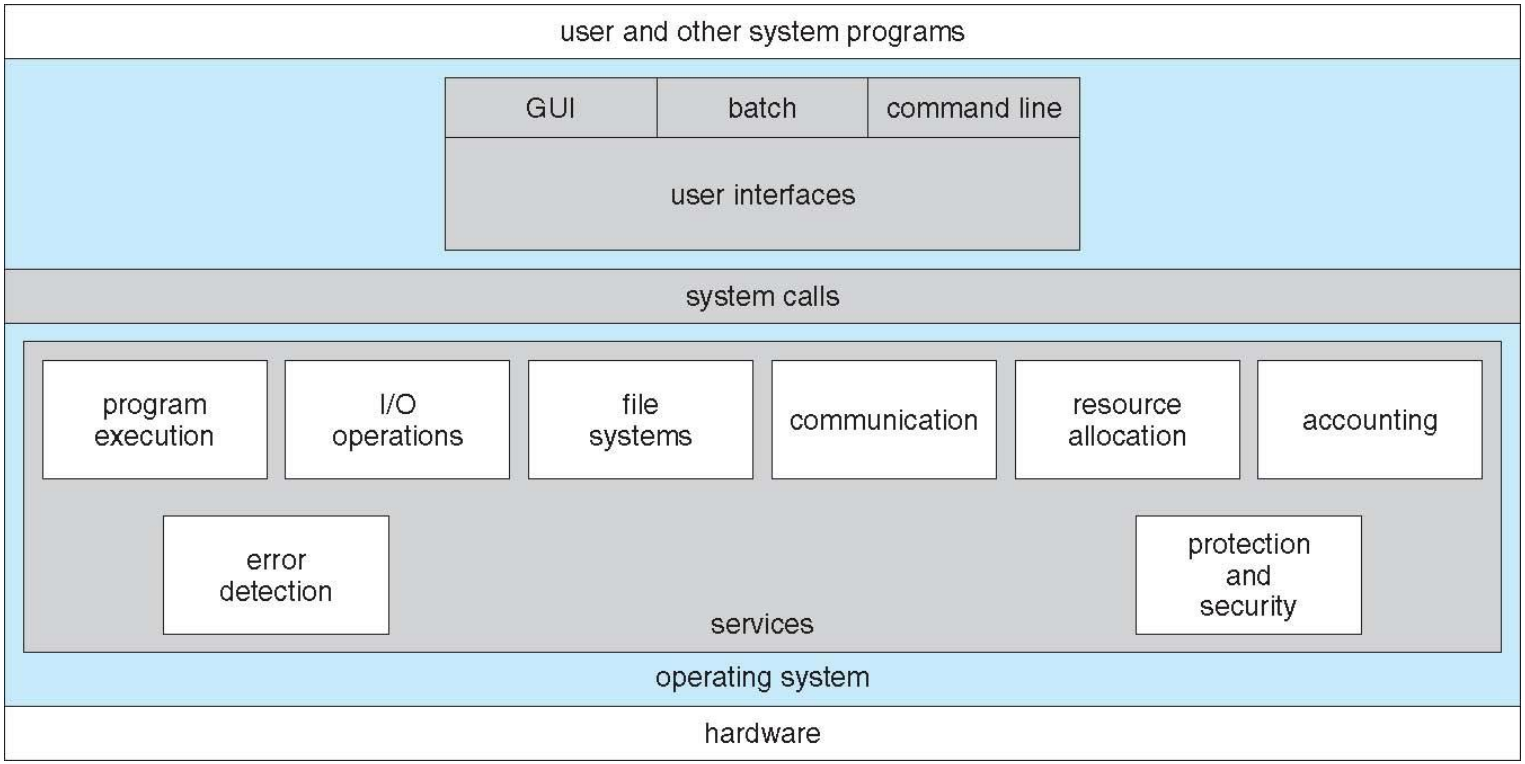


Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



A View of Operating System Services



User Operating System Interface - CLI

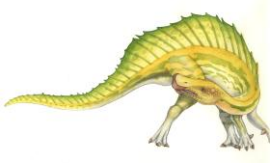
CLI or **command interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- E.gs create, delete, list, print, copy,execute

- Implemented in two ways
  - the command interpreter itself contains the code to execute the command.
    - E.g: a command to delete a file may cause the command interpreter to jump to a section of its code that sets up the parameters and makes the appropriate system call.
  - implements most commands through system programs
    - command interpreter does not understand the command in any way;
    - it merely uses the command to identify a file to be loaded into memory and executed
- If the latter, adding new features doesn't require shell modification

User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces

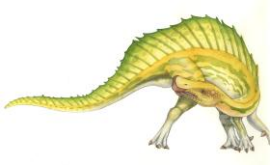


Touchscreen Interfaces

- Touchscreen devices require new interfaces
  - Mouse not possible or not desired
  - Actions and selection based on gestures
  - Virtual keyboard for text entry
- Voice commands.

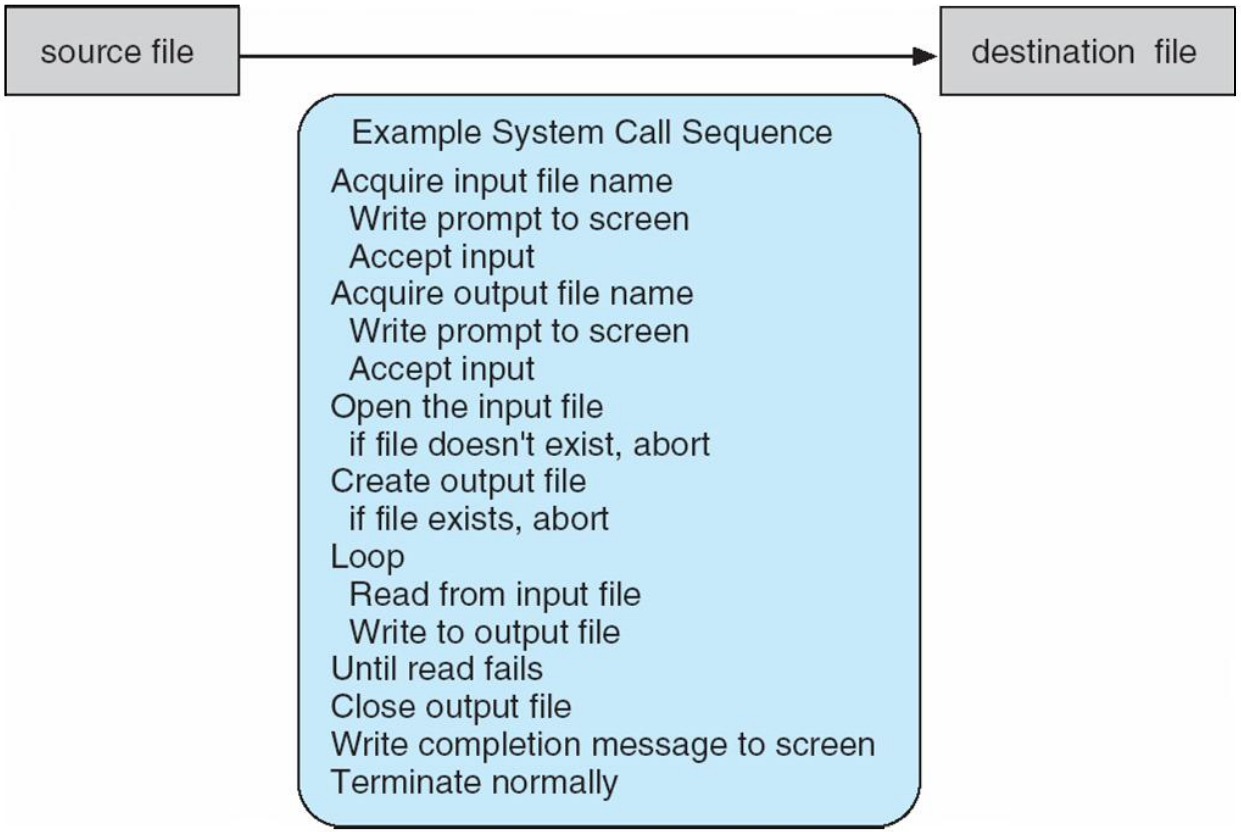
System Calls

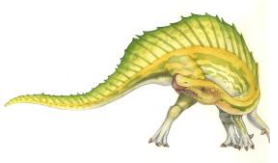
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- example to illustrate how system calls are used: writing a simple program to read data from one file and copy them to another file. The first input that the program will need is the names of the two files: the input file and the output file



Example of System Calls

- System call sequence to copy the contents of one file to another file





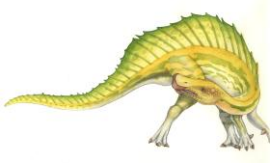
System Calls Contd..

- Most programmers never see this level of detail
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- the functions that make up an API typically invoke the actual system calls on behalf of the application programmer. For example, the Win32 function CreateProcess() (which unsurprisingly is used to create a new process) actually calls the NTCreateProcess() system call in the Windows kernel

Note that the system-call names used throughout this text are generic
Each operating system has its own name for each system call.

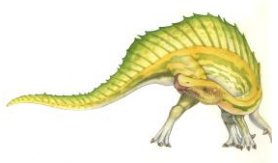
Reasons for programming according to an API rather than invoking actual system calls

- One benefit of programming according to an API concerns program portability: An application programmer designing a program using an API can expect her program to compile and run on any system that supports the same API (although in reality, architectural differences often make this more difficult than it may appear).
- Actual system calls can often be more detailed and difficult to work with than the API available to an application programmer.
- Regardless, there often exists a strong correlation between a function in the API and its associated system call within the kernel

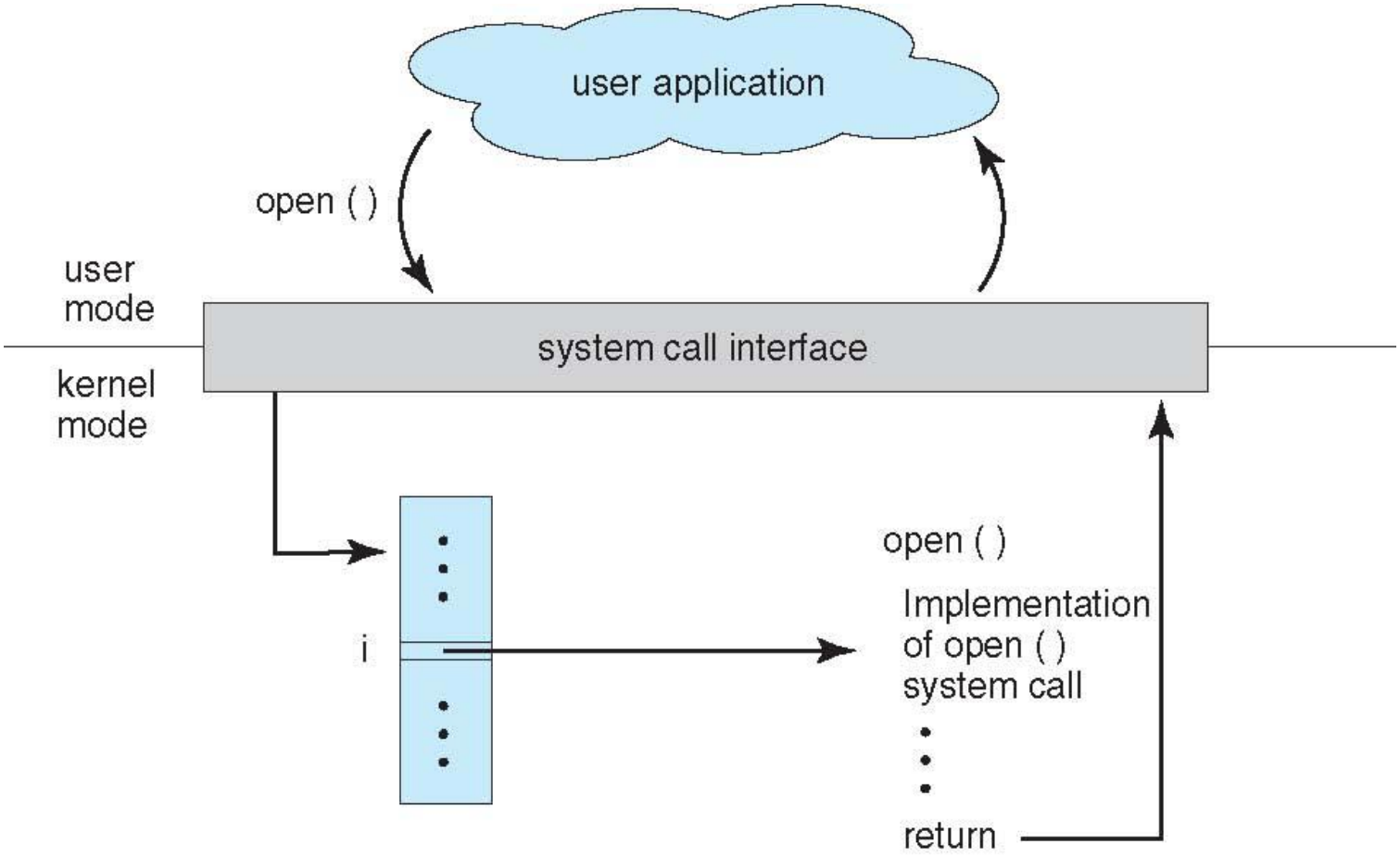


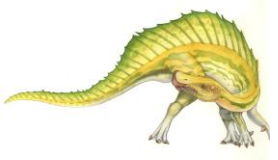
System Call Implementation

- The run-time support system (a set of functions built into libraries included with a compiler) for most programming languages provides a system-call interface that serves as the link to system calls made available by the operating system.
- The system-call interface intercepts function calls in the API and invokes the necessary system calls within the operating system
- Typically, a number associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)



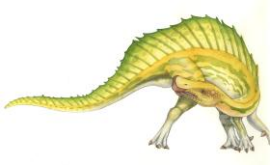
API – System Call – OS Relationship





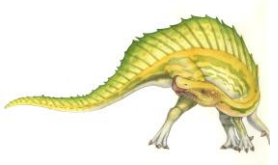
System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
  - For example, to get input, we may need to specify the file or device to use as the source, as well as the address and length of the memory buffer into which the input should be read.
  - Of course, the device or file and length may be implicit in the call.

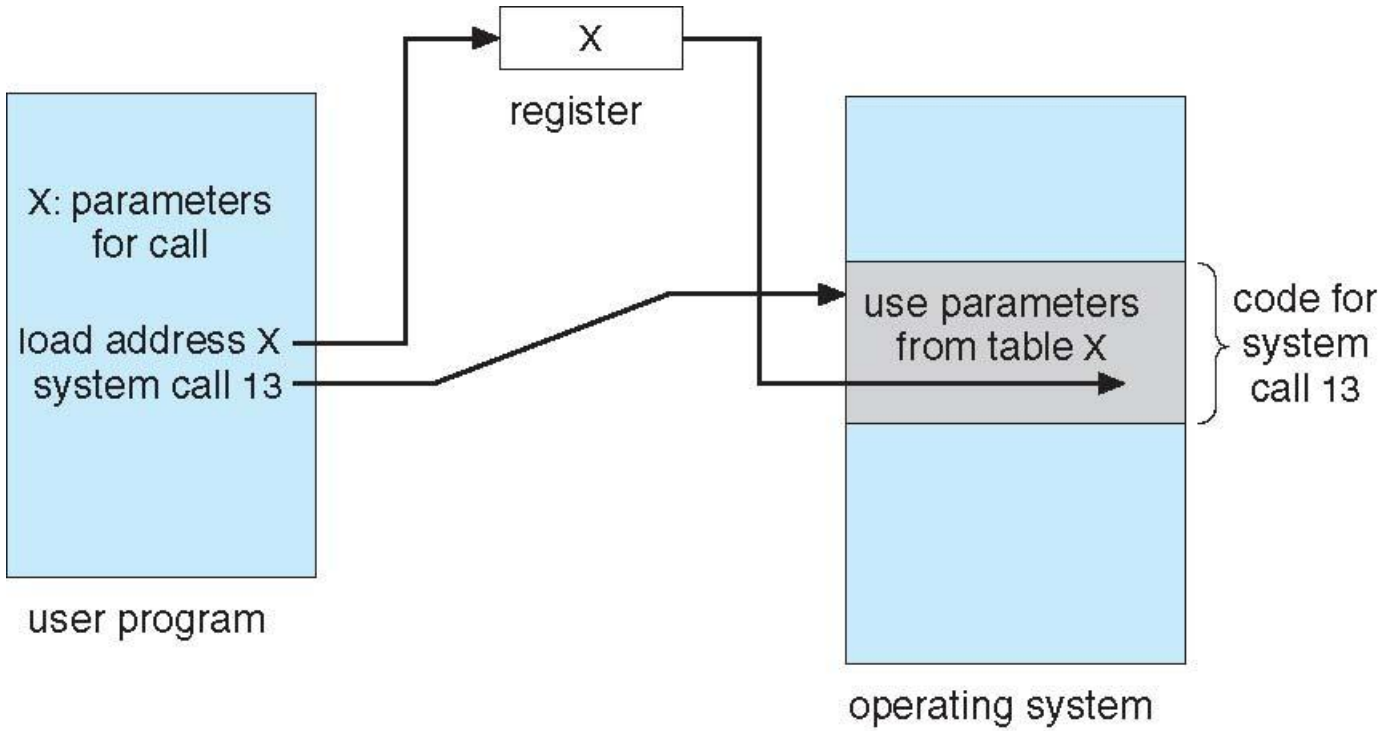


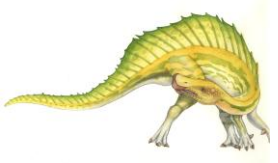
Passing parameters to the OS

- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed



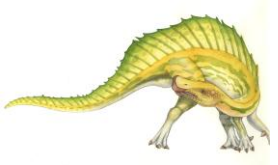
Parameter Passing via Table





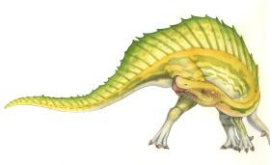
Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes



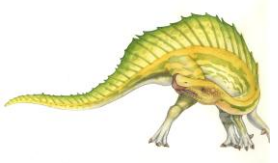
Types of System Calls

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices



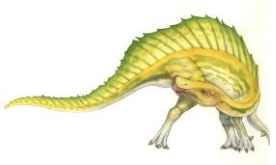
Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices



Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

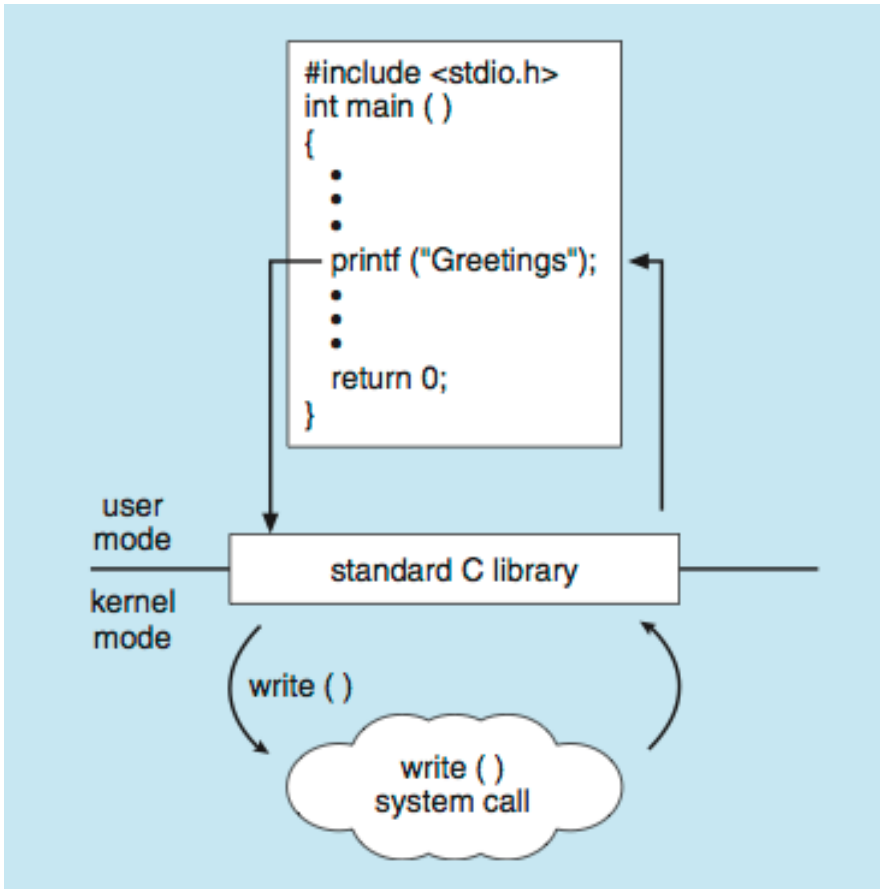


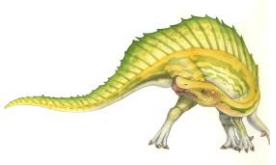
Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Standard C Library Example

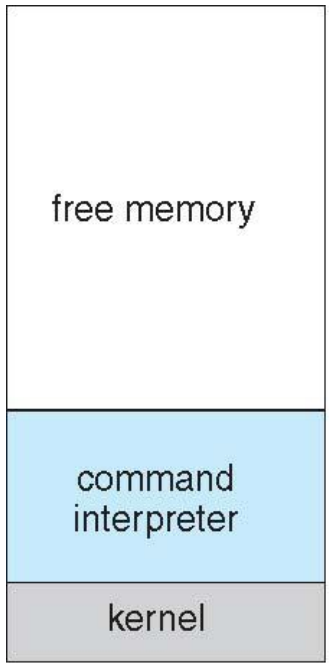
- C program invoking printf() library call, which calls write() system call





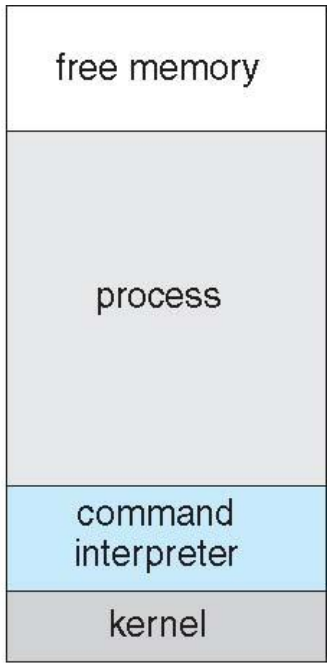
Example: MS-DOS

- Single-tasking
- Shell invoked when system booted
- Simple method to run program
  - No process created
- Single memory space
- Loads program into memory, overwriting all but the kernel
- Program exit -> shell reloaded



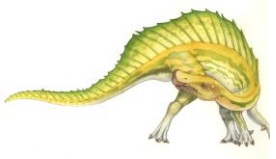
(a)

At system startup



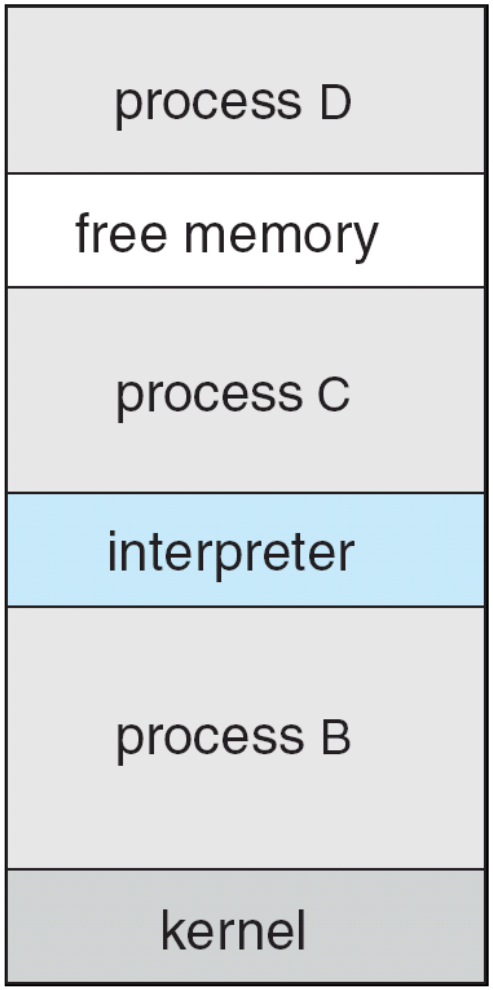
(b)

running a program



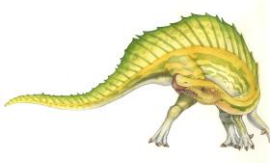
Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- Process exits with:
  - code = 0 – no error
  - code > 0 – error code



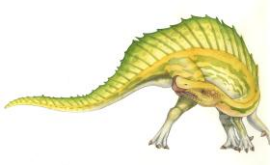
System Programs

- System programs also known as system utilities, provide a convenient environment for program development and execution.
- Some of them are simply user interfaces to system calls; others are considerably more complex
- They can be divided into:
  - File manipulation
  - Status information sometimes stored in a File modification
  - File modification
  - Programming language support
  - Program loading and execution
  - Communications
- Most users' view of the operation system is defined by system programs, not the actual system calls



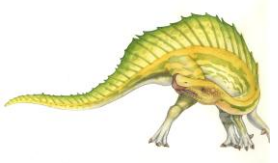
System Programs

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information



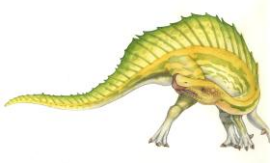
System Programs (Cont.)

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another



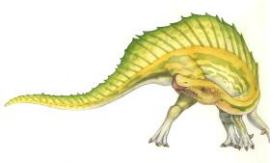
System Programs (Cont.)

- In addition to systems programs, most operating systems are supplied with programs that are useful in solving common problems or performing common operations.
- Such application programs include Web browsers, word processors and text formatters, spreadsheets, database systems, compilers, plotting and statistical-analysis packages, and games



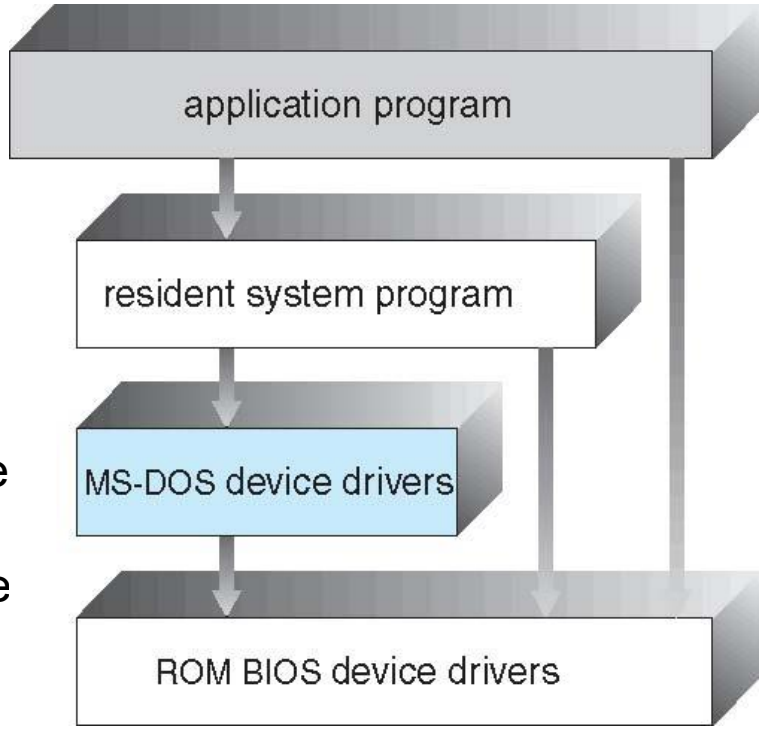
Operating System Structure

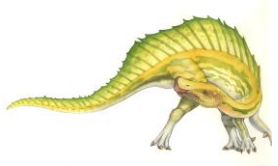
- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered – an abstrcation
  - Microkernel -Mach



Simple Structure -- MS-DOS

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated
- the interfaces and levels of functionality are not well separated.
- Application programs are able to access the basic I/O routines to write directly to the display and disk drives. Such freedom leave MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail.
- MS-DOS was also limited by the hardware of its era. Because the Intel 8088 for which it was written provides no dual mode and no hardware protection, the designers of MS-DOS had no choice but to leave the base hardware accessible

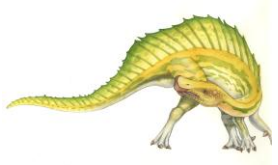




Non Simple Structure -- UNIX

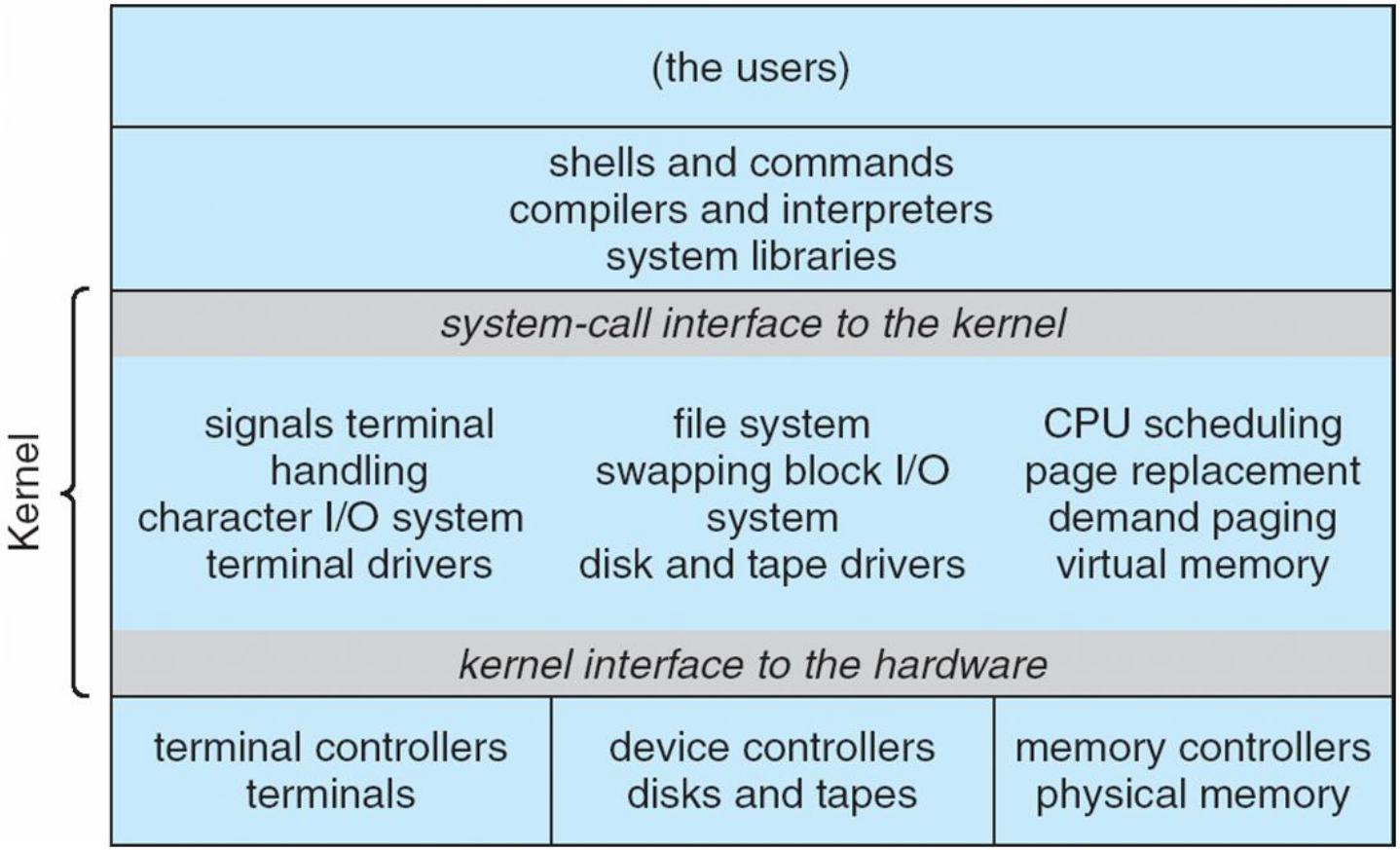
UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

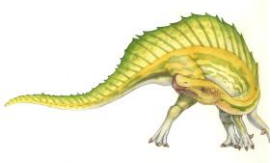
- Systems programs
- The kernel-is further separated into a series of interfaces and device drivers that have been added and expanded over the years as UNIX has evolved.
- can view the traditional UNIX operating system as being layered
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level



Traditional UNIX System Structure

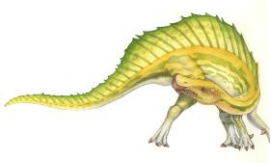
Beyond simple but not fully layered





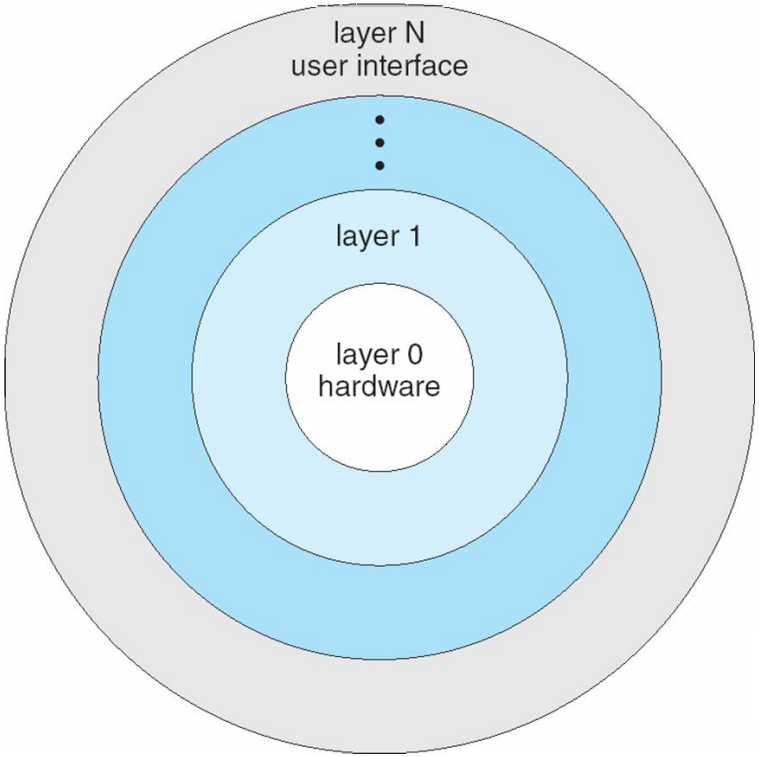
Layered Approach

- With proper hardware support, operating systems can be broken into pieces that are smaller and more appropriate than those allowed by the original MS-DOS and UNIX systems.
- The operating system can then retain much greater control over the computer and over the applications that make use of that computer.
- Implementers have more freedom in changing the inner workings of the system and in creating modular operating systems.
- Under a topdown approach, the overall functionality and features are determined and are separated into components.
- Information hiding is also important, because it leaves programmers free to implement the low-level routines as they see fit, provided that the external interface of the routine stays unchanged and that the routine itself performs the advertised task



Layered Approach Contd…

- A system can be made modular in many ways
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- An operating-system layer is an implementation of an abstract object made up of data and the operations that can manipulate those data.
- A typical operating-system layer—say, layer M—consists of data structures and a set of routines that can be invoked by higher-level layers. Layer M, in turn, can invoke operations on lower-level layers
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Layered Approach Contd…

- Advantage of the layered approach
  - simplicity of construction and debugging.
    - The layers are selected so that each uses functions (operations) and services of only lower-level layers.
    - simplifies debugging and system verification.
    - The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions.
    - Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on.
    - If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.
- Each layer is implemented with only those operations provided by lower level layers.
  - A layer does not need to know how these operations are implemented; it needs to know only what these operations do.
  - Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers

Layered Approach Contd…

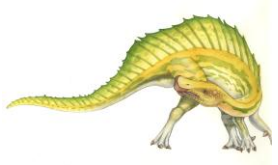
- Major difficulty with the layered approach
  - involves appropriately defining the various layers.
    - Because a layer can use only lower-level layers, careful planning is necessary.
    - For example, the device driver for the backing store (disk space used by virtual-memory algorithms) must be at a lower level than the memory-management routines, because memory management requires the ability to use the backing store
  - they tend to be less efficient than other types.
    - For instance, when a user program executes an I/O operation, it executes a system call that is trapped to the I/O layer, which calls the memory-management layer, which in turn calls the CPU-scheduling layer, which is then passed to the hardware.
    - At each layer, the parameters may be modified, data may need to be passed, and so on.
    - Each layer adds overhead to the system call; the net result is a system call that takes longer than does one on a nonlayered system

Microkernel System Structure

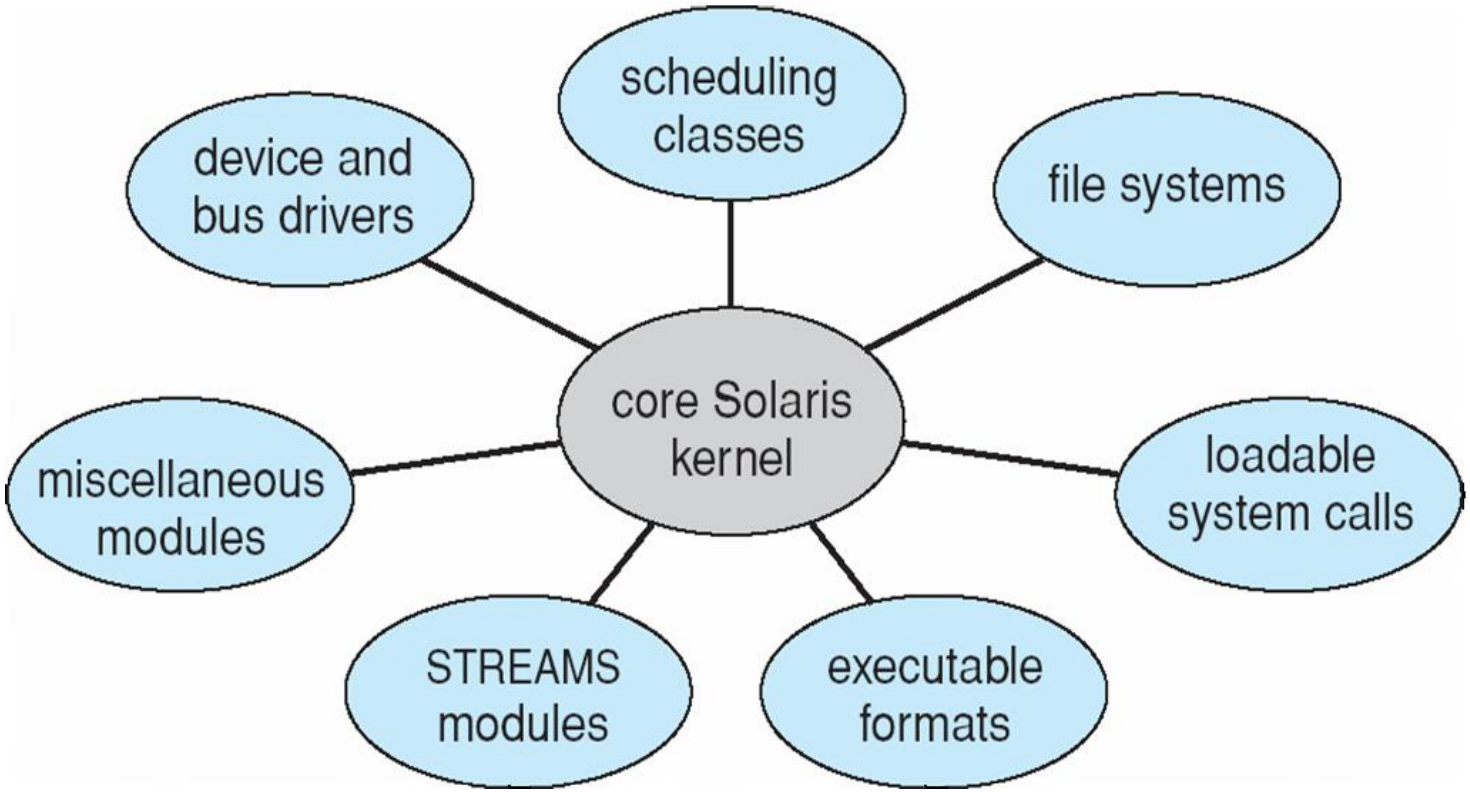
- structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs
- The result is a smaller kernel.
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel-All new services are added to user space and consequently do not require modification of the kernel.
  - Easier to port the operating system to new architectures
  - more security and reliability, since most services are running as user—rather than kernel—processes.
    - If a service fails, the rest of the operating system remains untouched
    - Detriments:
  - Performance overhead of user space to kernel space communication

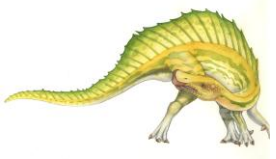
Modules

- Many modern operating systems implement **loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but more flexible than a layered system in that any module can call any other module.
- approach is like the microkernel approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules, but it is more efficient, because modules do not need to invoke message passing in order to communicate
  - Linux, Solaris, etc



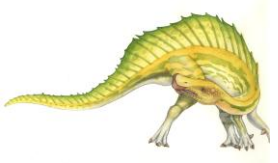
Solaris Modular Approach



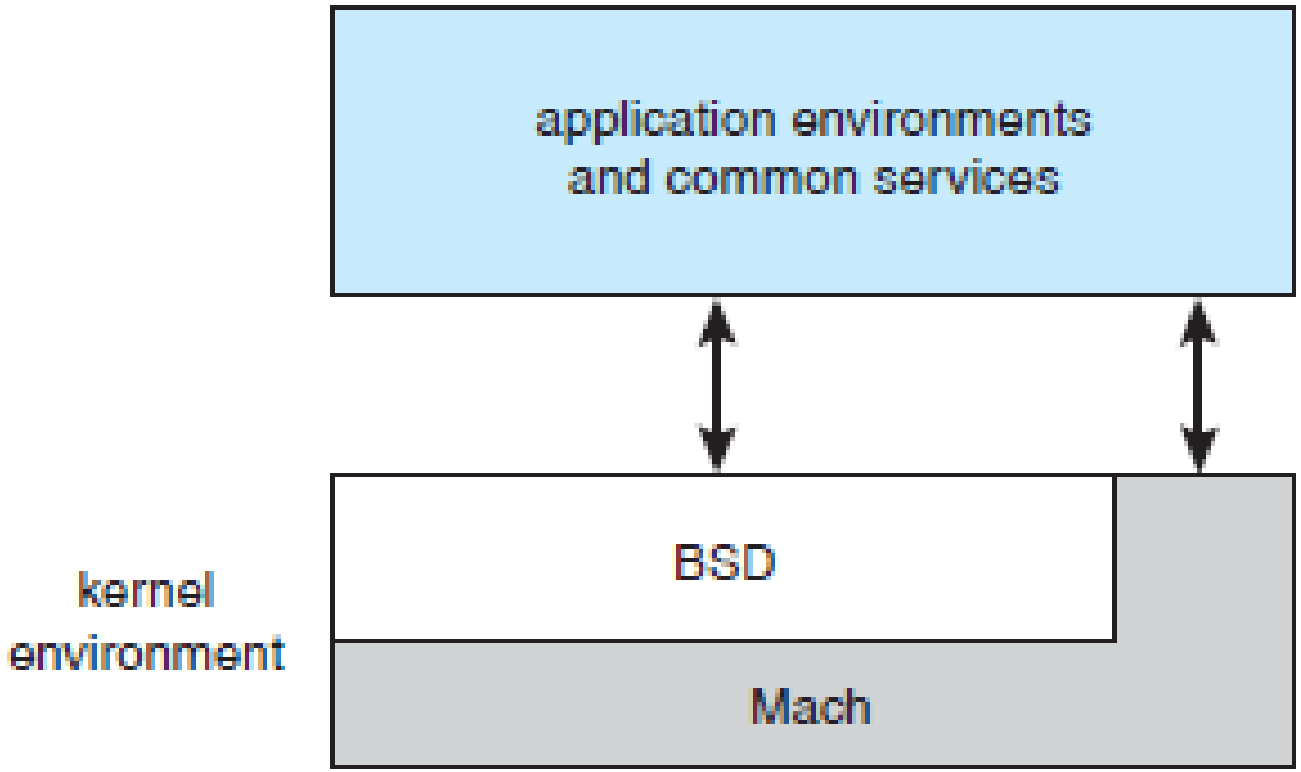


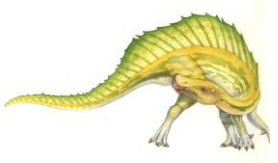
Hybrid Structure

- layered system in which one layer consists of the Mach microkernel.
- The top layers include application environments and a set of services providing a graphical interface to applications
- Below these layers is the kernel environment, which consists primarily of the Mach microkernel and the BSD kernel.
- Mach provides memory management; support for remote procedure calls (RPCs) and interprocess communication (IPC) facilities, including message passing; and thread scheduling.
- The BSD component provides a BSD command line interface, support for networking and file systems, and an implementation of POSIX APIs, including Pthreads



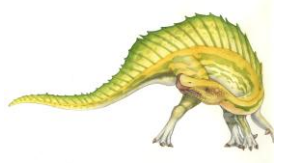
Mac OS X Structure



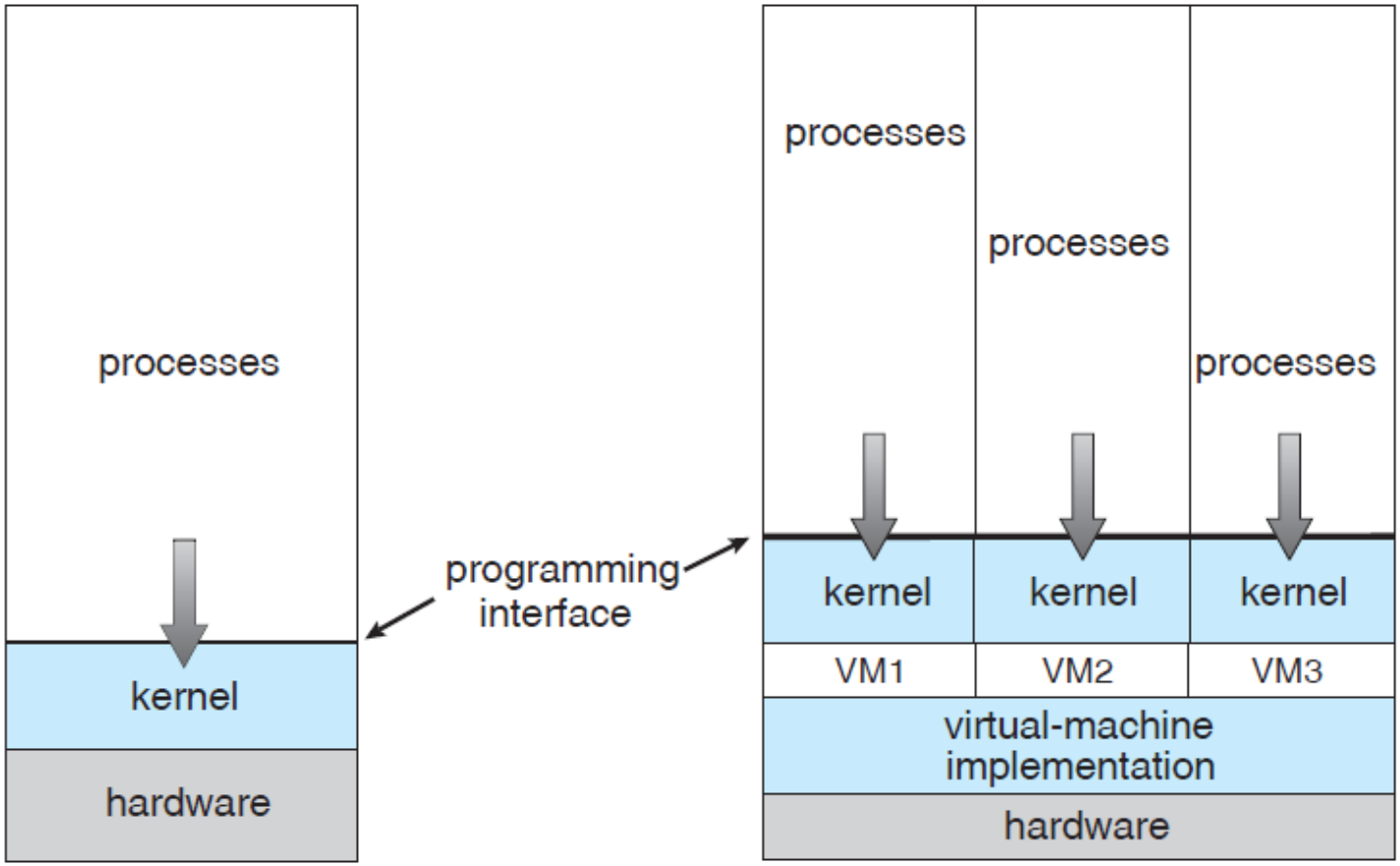


Virtual Machines

- abstract the hardware of a single computer (the CPU, memory, disk drives, network interface cards, and so forth) into several different execution environments, thereby creating the illusion that each separate execution environment is running its own private computer
- an operating system **host** can create the illusion that a process has its own processor with its own (virtual) memory.
- The virtual machine provides an interface that is *identical* to the underlying bare hardware.
- Each **guest** process is provided with a (virtual) copy of the underlying computer
- Guest process is in fact an operating system, and that is how a single physical machine can run multiple operating systems concurrently, each in its own virtual machine.



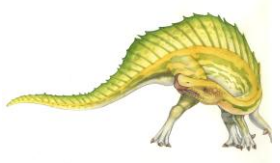
System Models- Non virtual mahineand Virtual Machine



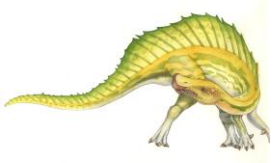
Benefits

- able to share the same hardware yet run several different execution environments
- the host system is protected from the virtual machines, just as the virtual machines are protected from each other
  - A virus inside a guest operating system might damage that operating system but is unlikely to affect the host or the other guests.
  - Because each virtual machine is completely isolated from all other virtual machines, there are no protection problems.
  - there is no direct sharing of resources.
    - Two approaches to provide sharing have been implemented.
    - First, it is possible to share a file-system volume and thus to share files.
    - Second, it is possible to define a network of virtual machines, each of which can send information over the virtual communications network.
    - The network is modeled after physical communication networks but is implemented in software

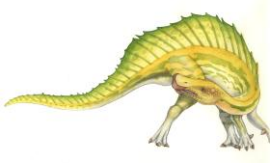
- A virtual-machine system is a perfect vehicle for operating-systems research and development.
- Normally, changing an operating system is a difficult task.
  - Operating systems are large and complex programs, and it is difficultmto be sure that a change in one part will not cause obscure bugs to appear in some other part.
  - The power of the operating system makes changing it particularly dangerous.
  - Because the operating system executes in kernel mode, a wrong change in a pointer could cause an error that would destroy the entiremfile system.
  - Thus, it is necessary to test all changes to the operating system carefully.



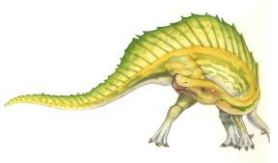
- The operating system runs on and controls the entire machine.
- Current system must be stopped and taken out of use while changes are made and tested. - *system development time*.
- Since it makes the system unavailable to users, system development time is often scheduled late at night or on weekends, when system load is low.
- System programmers are given their own virtual machine, and system development is done on the virtual machine instead of on a physical machine.
- Normal system operation seldom needs to be disrupted for system development.



- Multiple operating systems can be running on the developer's workstation concurrently.
- This virtualized workstation allows for rapid porting and testing of programs in varying environments.
- Similarly, quality-assurance engineers can test their applications in multiple environments without buying, powering, and maintaining a computer for each environment.

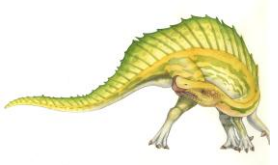


- A major advantage of virtual machines in production data-center use is system **consolidation**, which involves taking two or more separate systems and running them in virtual machines on one system.
- Such physical-to-virtual conversions result in resource optimization, as many lightly used systems can be combined to create one more heavily used system.
- application developers could pre-install the application on a tuned and customized operating system in a virtual machine.
- That virtual environment would be the release mechanism for the application
- application management would become easier, less tuning would be required, and technical support of the application would be more straightforward.
- Installation would be simple, and redeploying the application to another system would be much easier than the usual steps of uninstalling and reinstalling



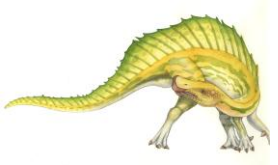
System Boot

- When power initialized on system, execution starts at a fixed memory location
  - Firmware ROM used to hold initial boot code
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – **bootstrap loader**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
- Common bootstrap loader, **GRUB**, (GRand Unified Bootloader) allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**



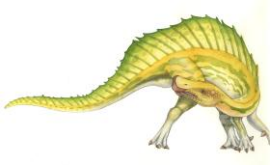
Acknowledgement

- Thie slides are borrowed from the ppt of Operating Systems Concepts 9Th Edition By Silberschatz, Galvin and Gagne with some modifications done



Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.



End of Chapter 2